

Enhancing Similarity Search Throughput by Dynamic Query Reordering

Filip Nalepa^(✉), Michal Batko, and Pavel Zezula

Faculty of Informatics, Masaryk University, Brno, Czech Republic
f.nalepa@gmail.com

Abstract. A lot of multimedia data are being created nowadays, which can only be searched by content since no searching metadata are available for them. To make the content search efficient, similarity indexing structures based on the metric-space model can be used. In our work, we focus on a scenario where the similarity search is used in the context of stream processing. In particular, there is a potentially infinite sequence (stream) of query objects, and a query needs to be executed for each of them. The goal is to maximize the throughput of processed queries while maintaining an acceptable delay. We propose an approach based on dynamic reordering of the incoming queries combined with caching of recent results. We were able to achieve up to 3.7 times higher throughput compared to the base case when no reordering and caching is used.

Keywords: Stream processing · Similarity search

1 Introduction

Current digital media explosion results in huge amounts of unstructured data that lack any searchable metadata. In order to make such data findable, the content-based search must be applied that treats the data by similarity rather than exact match of their attributes. Such search then usually uses *k-nearest-neighbors queries* (*kNN*), which retrieve the *k* objects that are the most similar to a given query object.

To make things even more complex, some applications need to deal with a continuous stream of arriving data that are to be searched. For example, consider a text search-engine crawler that gathers images from the web and needs to annotate them by textual descriptions according to the image content, a spam filter that compares the incoming emails to some learned spam knowledge base so that spam messages can be detected, or a news notification system that needs to compare the newly published articles to the profiles of all the subscribed users to find out who should be notified.

All these applications require to process each and every data item that appeared in the stream by some form of content-based searching. Luckily, the data in such applications need not be processed immediately but some small delay is acceptable. The most important thing is the number of processed images

in a given time, i.e. the *throughput* of such an application. The individual query search time can be improved by applying some similarity indexing technique, for which there are efficient algorithms based on the metric model of similarity [14]. As opposed to interactive applications focusing on the single query optimization, in our scenario, we can afford a slight delay of the single query processing if the overall throughput of the system is improved. Performance of such stream processing applications is studied in [7, 8].

In our work, we exploit the fact that the order of the processed queries may have a significant impact on the processing time. We propose a novel approach based on dynamic reordering of the incoming queries combined with caching mechanism used to lower the I/O costs. The solution is based on the assumption that to evaluate two similar queries, similar data of the index need to be accessed. According to our experiments, the proposed technique results in significantly higher throughput.

The rest of the paper is organized as follows. First, we present some related work on caching and stream data throughput improvements. In Sect. 3 we formally define our problem. Components and the overall architecture of our technique is presented in Sect. 4. Section 5 presents the basic query ordering technique that is further enhanced in Sect. 6 so that the approach can deal with too high delays. Experimental evaluation of our approach can be found in Sect. 7. Finally, the paper is concluded in Sect. 8.

2 Related Work

The usage of a caching mechanism in similarity search has been proposed in several papers to reduce the amount of disk accesses. In [3], the authors deal with kNN queries to search for similar images in the metric space. They build their approach on the assumption that there exists a set of popular images which are queried by users significantly more often than the other images. They propose an approach where the result sets of individual kNN queries are stored in a cache, and they are reused to produce approximate results of subsequent queries. Unlike traditional caching, the proposed cache can manage not only exact hits, but also approximate ones that are solved by similarity with respect to the result sets of past queries present in the cache.

The concept of caching in similarity search is used also in [10] where it is applied to contextual advertising systems. For a kNN query q , if there is a cache miss, a larger set of objects than are actually needed is retrieved from the disk and stored in the cache. When a similar query to the cached query q comes to the system, the cached values of q are explored to obtain results for the new query. In this way, an approximate answer is returned.

These two approaches use the cache to speedup the processing of a query for the price of reduced accuracy. On the other hand, our proposed approach does not compromise the accuracy, and the result for each query is exactly the same as it would be without the cache.

Another way to improve the throughput of a stream of kNN queries, is to reorder the queries. In [12], the authors optimize nearest neighbor search for

videos when each video is represented by a sequence of high-dimensional feature vectors. Given a query video containing n feature vectors, a search for each vector is performed, and the overall similarity is computed at last. The authors make use of the fact that nearby feature vectors in a video are similar, and they propose dynamic query ordering for advanced optimization of both I/O and CPU costs. They make an observation that the overlapped candidates of a previous query may help to further reduce the candidate sets of subsequent queries. The algorithm aims at progressively finding a query order such that the common candidates among queries are fully utilized to maximally reduce the total number of candidates.

The aforementioned techniques assume the existence of sequences of similar queries in the stream. In our approach, we use no such assumption, on the contrary, we reorder the queries so that we obtain the desired sequences.

A slightly different problem is dealt with in [13]. The stream of queries is given as a line segment in a space, and the task is to perform a continuous nearest neighbor search; that is to retrieve a nearest neighbor for every point on the line segment. The result contains a set of $(point, interval)$ tuples such that the *point* is the nearest neighbor of all points in the corresponding *interval*. The proposed technique uses a single database traversal to identify all the split points which form the result set. A motivating use case is to find all nearest gas stations during a route between two places.

Another task in similarity search for streams is a classification problem when a set of classes are assigned to each data item of the stream. Usually, the class labels are predicted based on some training data which are correctly labeled. The challenge in the stream processing is to deal with a high rate of incoming data items and with the concept drift, i.e., the applications have to adapt to new trends (new classes may emerge; some classes may disappear; the definition of the classes may change in time). In [15], there is a proposal of a new indexing structure called Ensemble-tree (E-tree) which is a height balanced tree consisting of R-tree like structure storing the decision rules and a table structure storing classifier-level information. Other approaches to the stream classification can be found in [4, 6, 11]. We do not address the problem of classification itself in our work; we focus rather on improving the efficiency of a particular type of classification (kNN classification).

3 Problem Definition

Suppose there is a domain of complex objects D (e.g., images) and a large database containing such objects. Let $s = (q_1, q_2, \dots)$ be a stream, i.e., a potentially infinite sequence of query objects of the same type, where $q_i \in D$ for each i . All the objects are indexed in the metric space which is a universal model of similarity [14]. There is defined a total distance function $d : D \times D \rightarrow R$. The distance between two objects corresponds to the level of their dissimilarity ($d(p, p) = 0$, $d(o, p) \geq 0$).

For each query object q_i in the stream s , a k-nearest neighbors query $NN(q_i, k)$ is executed which returns k nearest objects from the database to

the query object. The goal is to achieve as high throughput as possible, i.e., to process a given set of queries as fast as possible.

It is allowed to locally change the order of the processed queries. More precisely, we define a buffer B containing at maximum N query objects. A query object of the stream is added to the buffer only if it is not full, i.e., it contains less than N queries. As a next query to be processed, a query object from the buffer is selected.

Using the local reordering described above, some queries may be a subject to starvation, i.e., they never get selected from the buffer for processing. Therefore, we introduce a constraint on the *maximum delay* (MD) and update the retrieval method as follows. As soon as an object is stored to the buffer, it is assigned a timestamp. Whenever an object is about to be selected from the buffer and there exists an object whose timestamp is older than the allowed maximum delay, the object with the oldest timestamp in the buffer has to be processed.

Formally, a buffer $B = \{(q_1, t_1), \dots, (q_h, t_h)\}$ where $\{q_1, \dots, q_h\} \subseteq D$; $|B| \leq N$; t_i is the entrance time of q_i to the buffer. We define two functions for the buffer.

$push(q, t)$ inserts the query q to the buffer and assigns it the timestamp t .

$pop(timeLimit) = (q, t)$ removes the pair $(q, t) \in B$ from the buffer and returns it on the output. If there exists a pair $(p, u) \in B$ such that $u \leq timeLimit$, then the following has to hold: $t = \min(\{v | (r, v) \in B\})$.

The generic processing of the stream of queries is performed as follows. The buffer is filled with first N queries of the stream. Then there is a cycle in which a query is popped from the buffer, it is processed, and another query is pushed into the buffer. It means that there are always N queries in the buffer before a query is popped. The pseudocode can be seen in Algorithm 1.

Algorithm 1. Generic algorithm

```

function processStream( $(q_1, q_2, \dots), N, MD$ )
   $B \leftarrow \text{new } Buffer()$ 
   $i \leftarrow 1$ 
  while  $i \leq N$  do
     $B.push(q_i, now())$ 
     $i++$ 
  loop
     $process(B.pop(now() - MD))$ 
     $B.push(q_i, now())$ 
     $i++$ 

```

To measure the throughput, we define the function $processingTime((q_1, q_2, \dots), N, m) = T$ where T is the time since the algorithm was launched until m queries have been processed, i.e., until $i = N + m + 1$ in Algorithm 1. By repeatedly calling pop , a permutation $(q_{i_1}, q_{i_2}, \dots)$ of the original stream is continuously generated. The ultimate goal is to specify the function pop so that T

is minimized. Note that *pop* can access only the queries which are in the buffer, and it cannot foresee any future ones.

4 Architecture

In this section, we describe the architecture of the whole system used for processing a stream of similarity search queries. There are two main parts: the metric index and the buffer of waiting queries. The schema is shown in Fig. 1.

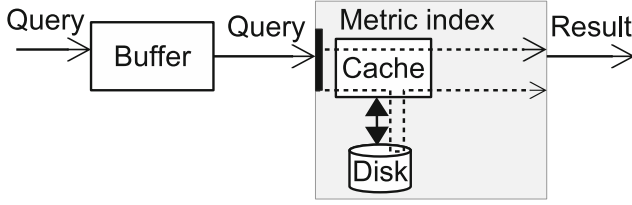


Fig. 1. Architecture

The buffer is used to temporarily store the incoming query objects which are waiting for processing. The metric index takes care of the query evaluation. It contains a disk where the database of objects is stored and a main memory cache used for storing recently loaded data from the disk.

Whenever there is a spare space in the buffer, the next query of the stream is loaded. When the metric index is ready for processing another query, a query is picked from the buffer according to a chosen strategy. During processing of the query, the metric index looks into the cache to possibly use any intermediate results obtained from evaluating recent queries. If the data are not in the cache, they are loaded from the disk.

We consider a generic metric index which uses data partitioning $P = \{p_1, \dots, p_n\}$ where $p_i \subseteq D$. When evaluating a query, it needs to access a subset of the partitions $Q \subseteq P$. The partitions are typically stored on a disk [14].

The cache system is based on storing intermediate results of recent queries so that they can be reused by the metric index when processing similar queries. The cache is used to lower down the number of disk accesses when retrieving data partitions. The loaded partitions are kept in the cache so that they can be reused by later queries which access the same partitions. The cache is a set of partitions $cache = \{p_1, \dots, p_m\} \subseteq P$. The size of the cache is limited by the number of objects within the cached partitions: $\sum_{p \in cache} |p| \leq cacheLimit$. To measure the utility of the cache, we define the function

$$cacheUtility(q, cache) = \frac{|I(q) \cap cache|}{|I(q)|} \quad (1)$$

where *cache* represents the content of the cache and $I(q) \subseteq P$ is the set of partitions accessed during the evaluation of q .

To keep track of the content of the cache, we define the function $updateCache(q, cache)$ returning the content of the cache after processing the query q where *cache* represents the content of the cache before executing q . In our implementation, we use the least recently used policy. In particular, the partitions with the oldest last access time are discarded and replaced with the new partitions of the last query while obeying *cacheLimit*.

The $queryTime(q, cacheUtility)$ represents the time to process the given query q using the given cache utility. The desired property of the function is that it should be decreasing with increasing cache utility. (The validity of the assumption is verified experimentally in Sect. 7.2.)

$$cu_1 \leq cu_2 \Leftrightarrow queryTime(q, cu_1) \geq queryTime(q, cu_2) \quad (2)$$

where cu_1, cu_2 are cache utility.

Let us get back to the *processingTime* function which is introduced in Sect. 3 to specify the function more precisely with the use of the cache.

$$processingTime((q_1, \dots), N, m) = \sum_{j=1}^m (queryTime(q_{i_j}, cu_j)) + bt \text{ where} \quad (3)$$

$$cu_j = cacheUtility(q_{i_j}, cache_{j-1})$$

$$cache_0 = \{\}$$

$$cache_j = updateCache(q_{i_j}, cache_{j-1}) \text{ for } 1 \leq j \leq m$$

$(q_{i_1}, \dots, q_{i_m})$ is the sequence of queries generated by the *pop* function. $cache_j$ for $1 \leq j \leq m$ represents the state of the cache right after q_{i_j} has been processed. bt is the overall time needed for the buffer management (*push* and *pop* calls).

Let us denote $sequenceTime((q_{i_1}, \dots, q_{i_m})) = \sum_{j=1}^m (queryTime(q_{i_j}, cu_j))$. It is likely that $sequenceTime(s) \gg bt$ since the evaluation of a query is a costly operation. Therefore we will focus on minimization of *sequenceTime* in the next section.

5 Query Ordering

In this section, we discuss the optimal query ordering so that the minimal processing time is achieved, i.e., the throughput is maximized.

According to Formula 2 showing the dependency of the query time on the cache utility, we can suppose that the key to a good ordering of the queries is to maximize the cache utility. As for metric indexes, the following formula typically holds [14]:

$$d(q_1, q_2) \approx 0 \rightarrow I(q_1) \approx I(q_2) \quad (4)$$

That is, if the queries q_1, q_2 are close to each other, the sets of accessed partitions are approximately the same. In other words, if q_1 is processed, the $I(q_1)$ is stored in the cache, and subsequently q_2 is processed, then the cache utility is likely to be very high. The idea is to find such a sequence of queries where the neighboring ones are similar.

Let us start by defining a function returning the number of pairs of similar subsequent queries.

$$\text{similarCount}((q_1, \dots, q_m)) = |\{i | d(q_{i-1}, q_i) \approx 0 \wedge 2 \leq i \leq m\}| \quad (5)$$

Our assumption is that by maximizing *similarCount*, we can achieve high cache utility and thus low processing time.

Let the function *overallCacheUtility* return the fraction of all the partitions retrieved from the cache out of all the requested partitions for a given sequence of queries.

$$\text{overallCacheUtility}((q_1, \dots, q_m), \text{initialCache}) = \frac{\sum_{i=1}^m |I(q_i) \cap \text{cache}_{i-1}|}{\sum_{i=1}^m |I(q_i)|} \quad (6)$$

where $\text{cache}_0 = \text{initialCache}$ and cache_i is the content of the cache after processing q_i for $1 \leq i \leq m-1$.

Let r and s be sequences of queries of the same lengths; we assume the following to hold

$$\begin{aligned} \text{similarCount}(r) \leq \text{similarCount}(s) &\Rightarrow \\ \text{overallCacheUtility}(r, \{\}) &\leq \text{overallCacheUtility}(s, \{\}) \Rightarrow \\ \text{sequenceTime}(r) &\geq \text{sequenceTime}(s) \end{aligned} \quad (7)$$

Let us define the function returning the indexes of the queries which are not similar to the previous query in the given sequence $s = (q_1, \dots, q_m)$.

$$\text{dissimilar}(s) = (x_1, \dots, x_h) \quad (8)$$

so that x_i is in the tuple iff $x_i = 1 \vee d(q_{x_i}, q_{x_{i-1}}) \not\approx 0$. The function returns the elements of the tuple in the ascending order, i.e., $x_{i-1} < x_i$ for each i .

Now, the number of similar pairs can be computed by summing the number of queries between the dissimilar pairs:

$$\text{similarCount}(s) = \sum_{i=1}^h (x_i - 1 - x_{i-1}) + (m - x_h) \quad (9)$$

where $x_0 = 0$.

Let us denote $\text{simSeq}_i = x_i - 1 - x_{i-1}$. Then

$$\text{similarCount}(s) = \sum_{i=1}^h \text{simSeq}_i + (m - x_h) \quad (10)$$

Each $simSeq_i$ represents the number of subsequent pairs of similar queries. To achieve high *similarCount*, we propose a greedy algorithm which for each i tries to maximize $simSeq_i$.

First, the algorithm selects a highly populated region of query objects in the buffer. Subsequently, it processes all the queries in this region. By operating in dense regions, there is an increased chance of finding a long sequence of similar queries, hence achieving large $simSeq_i$.

A challenge is to efficiently search for dense regions. To achieve this, we cluster all the queries in the buffer based on their mutual distances in the metric space, i.e., any two queries in the same cluster should be close to each other. Therefore the task to find dense regions is reduced to searching for clusters containing large number of queries. The clustering has to be done on the fly as soon as a query enters the buffer and it is required not to impose a significant overhead so that it does not actually slow down the whole process. Formally, the cluster $C = \{(q_1, t_1) \dots, (q_m, t_m)\} \subseteq B$ where $d(q_i, q_j) \approx 0$ for each i, j . It is reasonable to adapt the clustering technique to the used indexing mechanism. In our case, an adequate clustering technique is a pivot based approach [9]. Specifically, there is a set of reference objects (pivots). To assign a query to a cluster, the distances of the query object to all the pivots are computed. The pivots are then sorted by the distances and the obtained pivot permutation determines the cluster.

The pseudocode can be seen in Algorithm 2. The *pop* function is called repeatedly according to Algorithm 1. Its functionality depends on a state which keeps a reference to the content of the buffer and to the currently processed cluster. If there are no more query objects in *currentCluster*, it finds the most populated cluster of query objects in the buffer and sets it as *currentCluster*. Then a query object is taken from *currentCluster* and returned to be processed.

Algorithm 2. Greedy algorithm

```

var buffer, currentCluster
function pop
  if empty(currentCluster) then
    currentCluster  $\leftarrow$  getMostPopulatedCluster(buffer)
  query  $\leftarrow$  currentCluster.getNext()()
  removeFromBuffer(buffer, query)
  return query

```

6 Delay Limit

In the definition of the problem, we have defined a limit for the maximum delay. Using the strategy for query ordering as stated above, there can be clusters which do not ever get to be processed. This is the case of clusters with low number of queries that are never selected as the cluster with the highest population. In this section, we show how to modify the query ordering so that the delay limit is obeyed.

To remind the delay limit, whenever a query is about to be picked from the buffer and there exists a query whose timestamp is older than the allowed delay, the query with the oldest timestamp in the buffer has to be processed.

Since a timed out query may be dissimilar to the previous queries, it is probable that the cache utility will be zero. The situation can be observed in Fig. 2b. The processing of the cluster A is interrupted by the timed out query in the cluster B . The question is how to proceed after processing the timed out query. Basically, there are two options. The first one is not to cache the results of the timed out query and continue with the processing of the cluster A . The second option is to cache the results of the timed out query and continue with the processing of the cluster B . According to the proposed greedy algorithm, we should select the option which maximizes the number of consecutive queries having small distances between them (maximizing $simSeq_i$, see Formula 10).

The option can be selected based on the number of query objects in the currently processed cluster and in the cluster containing the timed out query. Using the approach presented in the previous section, the cluster with the bigger number of queries should be chosen. However, selecting always the most populated cluster does not guarantee the sequence of nearby queries is maximized because processing of the queries may be interrupted by a timed out query from another cluster as we have seen in Fig. 2b. Therefore it is needed to consider also the timeouts which may occur during processing of a cluster.

The key question is how many queries in the selected cluster can be processed without any interruption, i.e., without encountering any dissimilar query object (x_i in Formula 8). The interruption happens in two situations: all the queries in the currently processed cluster are finished or there is a timeout outside the selected cluster. Therefore the number of subsequent similar queries is the minimum of the number of the queries in the cluster and the number of the queries which can be processed until the first timeout occurring in a different cluster.

To implement the greedy algorithm, we compute the time after which the processing of a given cluster would be interrupted. The algorithm selects the cluster with the largest time before an interruption. Using this approach, we can prioritize small clusters with early timeouts over large clusters with late timeouts.

Let B_t be the content of the buffer at time t and the cluster

$$C = \{(q_1, t_1), \dots, (q_m, t_m)\} \subseteq B_t$$

For the sake of simplicity, let us suppose the query times are constant after processing the first query in the cluster since the cache utility is constant, i.e., $queryTime(q_i, cacheUtility) = c$ for $2 \leq i \leq m$, and $queryTime(q_1, 0) = d$. Then

$$clusterTime(C) = sequenceTime((q_1, \dots, q_m)) = c \cdot (|C| - 1) + d \quad (11)$$

is the time needed to process the cluster C .

Now we derive the time after which there is a timeout outside the cluster C . Let MD be the maximum delay limit.

$$earliestTimeout(B_t, C) = \max\{0, \min\{MD - (t - e) \mid (q, e) \in B_t \wedge (q, e) \notin C\}\} \quad (12)$$

is the earliest time after which there is a timeout outside the cluster C . Note that all the computations are related to the state of the buffer at the time t since we cannot predict what queries will enter the buffer after t .

When put together, the time after which the processing of the cluster C is interrupted is $\min\{clusterTime(C), earliestTimeout(B_t, C)\}$. The greedy algorithm selects the cluster which maximizes the interruption time.

The pseudocode can be found in Algorithm 3. If there exists a query object exceeding the delay limit, the oldest query in the buffer is chosen to be processed. The *currentCluster* and *timedOutCluster* are compared regarding the time until an interruption. If *currentCluster* has a larger time, the results of the timed out query are not cached since its cluster will not be processed in the next round and it would be useless. If there is no timeout and if *currentCluster* does not contain any more queries, a new cluster maximizing the time until interruption is selected from all the clusters in the buffer. Then a next query is taken from *currentCluster*; it is removed from the buffer, and processed. The *pop* function returns also the information whether the caching should be used during processing of the returned query. The individual situations are illustrated in Fig. 2.

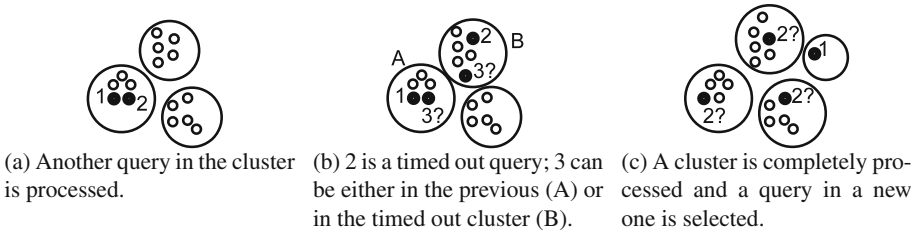


Fig. 2. Algorithm illustration; the numbers indicate the order of processed queries

Since Algorithm 3 can disable caching for a particular query, we have to slightly modify the definition of *similarCount* so that it skips the queries for which the caching was disabled.

$$similarCount((q_1, \dots, q_m)) = |\{i \mid d(q_{prevCached(i)}, q_i) \approx 0 \wedge 2 \leq i \leq m\}| \quad (13)$$

where $prevCached(i) = \max\{h \mid h < i \wedge (caching(q_h) \vee h = 1)\}$ and $caching(q)$ is true iff the caching was enabled when processing q .

Note that in case of a timed out query, we choose the next *currentCluster* from only two clusters, so it is in compliance with the greedy approach. Processing of the timed out query outside *currentCluster* generates an item

Algorithm 3. Full greedy algorithm considering the delay limit

```

var buffer, currentCluster
function pop(timeLimit)
  caching  $\leftarrow$  true
  if existsTimeout(buffer, timeLimit) then
    query  $\leftarrow$  getOldestQuery(buffer)
    timedoutCluster  $\leftarrow$  getClusterByQuery(query)
    currentCluster  $\leftarrow$  findClusterWithLargestTimeUntilInterruption()
  [currentCluster, timedOutCluster]
    if timedOutCluster  $\neq$  currentCluster then
      caching  $\leftarrow$  false
  else
    if isEmpty(currentCluster) then
      currentCluster  $\leftarrow$  findClusterWithLargestTimeUntilInterruption()
  buffer.getClusters()
    query  $\leftarrow$  currentCluster.getNext()
  removeFromBuffer(buffer, query)
  return query, caching

```

in *dissimilar* (Formula 8) since $d(q_i, q_{i-1}) \not\approx 0$. We either cache the results of the processing and proceed with processing the cluster of the timed out query, hence using the cached values, or the results are not cached and we get back to the original *currentCluster* and use the already cached values. If a different cluster was selected, there would be $dissimilar(s) = (\dots, i, i+1, \dots)$ where i corresponds to the timed out query and $i+1$ corresponds to the first query of the new cluster. This would violate the greedy algorithm since there would be a zero-length sequence of similar items ($simSeq_k = 0$ in Formula 10 for the corresponding k).

Since the greedy algorithm always chooses just one cluster which maximizes the cache utility at that moment, it can be very efficient. On the other hand, the best global ordering of the queries may be missed that way, and algorithms which compute the best ordering of all the queries in the buffer could be considered. However, they are likely to be slower and since there are new queries continuously added to the buffer, the computed ordering is only relevant for the actual state of the buffer.

7 Experiments

In this section, we experimentally validate the hypotheses presented above.

7.1 Experiment Setup

We use the M-Index [9] technique to index the metric-space data. It employs practically all known principles of metric space partitioning, pruning, and filtering, thus reaching high search performance. The actual data are partitioned into

buckets which are stored as separate files on a disk and read into the main memory during query evaluations. To partition the data, M-Index uses a set of pivots. To insert an object into the index, the pivots are sorted based on the distance to the object. In this way, a pivot permutation is obtained which identifies the bucket to insert the object. During a similarity search, mutual distances between the query object and the pivots are used to reduce the set of buckets which need to be accessed. The M-Index supports (among others) executing approximate kNN queries. One of the stop conditions of a query evaluation is given by the maximum number of accessed objects (the size of a candidate set). Such a stop condition is used in our experiments.

We use the Profimedia dataset of images [2] in the experiments. We created three different subsets of the images and extracted their visual-feature descriptors. The generated datasets are: 1 million Caffe descriptors [5] (4096 dimensional vectors), 1 million MPEG-7 descriptors [2] and 10 million MPEG-7 descriptors. Separately, we created streams of images represented by corresponding descriptors. During each experiment, the image descriptors of the stream are continuously sent to the buffer and processed by an approximate 10-NN query.

As the streamed queries enter the buffer, they are clustered using the pivots of the M-Index as described in Sect. 5. The clustering does not introduce a significant overhead since the same set of pivots is used also during the query evaluation and the computed distances can be reused.

The tested applications are implemented using Java programming language with the use of the MESSIF library [1] providing an implementation of the M-Index. The experiments were run on Intel Xeon 2.00 GHz with 8 GB RAM. The descriptors of the datasets are stored on a HDD.

7.2 Cache Utility

Figure 3a shows results of our experiments exploring the impact of the cache utility on the query time. We ran approximate 10-NN queries for each dataset and we were continuously changing the percentage of accessed buckets stored in the cache. The M-Index used candidate sets of size 10,000. The x-axis shows the percentage of the cached values; the y-axis represents the percentage of the time to process the query compared to the situation when the cache is not used. It can be observed that the processing time can be improved dramatically if the cache is filled with appropriate values, thus the assumption in Formula 2 is valid.

7.3 Buffer Size

In this group of experiments, we explore the impact of the size of the buffer. The experiments were conducted for all three datasets of descriptors. No maximal delay constraint was used. The maximum size of the cache was set to 90,000 descriptors for the 10 mil. MPEG-7 dataset (i.e., 0.9% of the database); up to 40,000 descriptors were stored for the 1 mil. MPEG-7 and Caffe datasets (i.e., 4% of the database). The size of the M-Index candidate set was 2,000 for the 10 mil. MPEG-7 dataset, and 1,000 for the others. With a growing size of the buffer, also

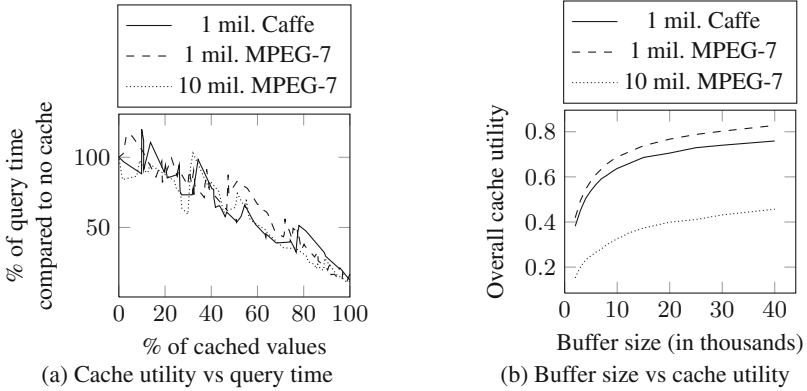


Fig. 3. Cache utility experiments

the cache utility and the throughput grow since the processed clusters are more populated and the cached values are reused more times, i.e., *similarCount* is higher, see Figs. 3b and 4a. The throughput speedup was computed as the ratio of the number of processed queries using a given buffer size and the number of processed queries without the usage of the proposed optimizations. The increase in the median delay of the processed queries (the time spent in the buffer) is shown in Fig. 4b. It was also measured there are just minor overhead costs connected with the buffer management (query clustering and reordering).

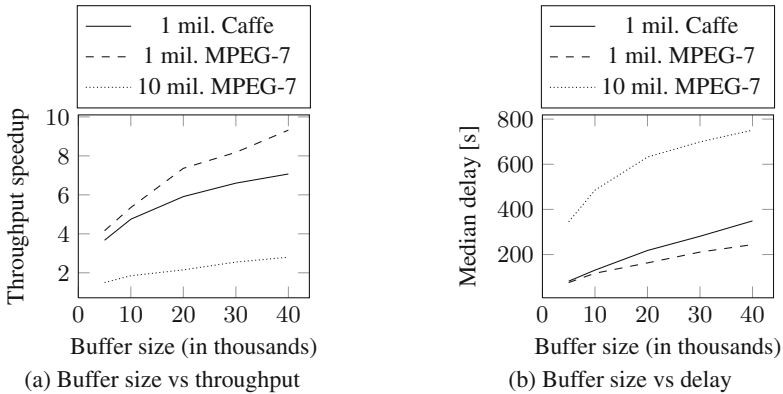


Fig. 4. Buffer size analysis; every experiment ran 30 min

7.4 Greedy Algorithm

In this section we evaluate four different approaches to processing a stream of kNN queries, and we experimentally prove Formula 7, i.e., by greedily

maximizing the number of similar pairs of queries, we achieve high cache utility and low processing time.

The first approach is the *base* (B) algorithm when the queries are processed by the M-Index one by one without any stream optimizations. The second one is the *densest first* (DF) algorithm. The queries are clustered, and the greedy algorithm always selects the most populated cluster. In case of a timeout, the timed out query is processed and the processing returns back to the previous cluster. In the third approach called *densest first with timeouts* (DFT), the greedy algorithm again chooses the most populated cluster. But after a timed out query is processed, it decides between the previous cluster and the cluster containing the timed out query. It picks the more populated one of them to continue with the processing. In the fourth approach, the *full* (F) greedy algorithm is used. Specifically, it maximizes the sequences of similar queries by computing the time until interruption for individual clusters (Algorithm 3).

In the first group of experiments, we used the dataset of 1 mil. Caffe descriptors. The approaches using the cache (DF, DFT, F) stored up to 40,000 descriptors in the cache (4 % of the DB) and used a buffer of size 8,000. The delay limit was set to 15 min, and 100,000 queries were processed for each approach. The size of the candidate set of the M-Index was set to 1,000 for each approximate kNN query.

See Fig. 5a for the results. The buffer-based approaches were able to process the queries much faster than the *base* algorithm. The individual optimizations pay off as expected, and the best results are obtained using the *full* greedy algorithm when the processing time was reduced 3.69 times compared to the *base* approach. The impact of individual optimizations can be observed also in the number of timed out queries: DF with 13408 timeouts vs F with 1491 timeouts. The cache utility of the *full* greedy algorithm was 0.62; its median delay (the time since a query enters the buffer until it is processed) was 128 s.

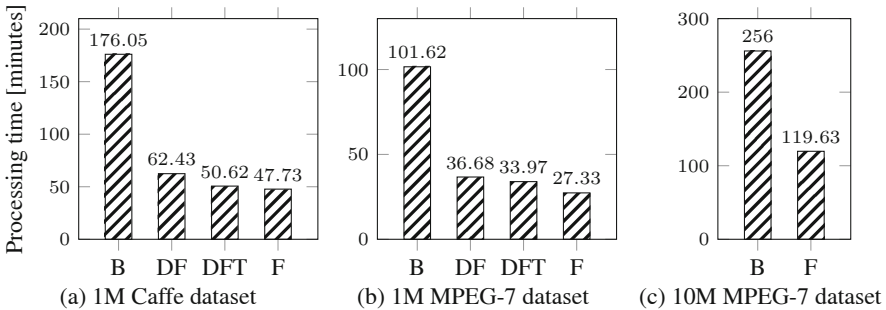


Fig. 5. Processing times of 100,000 queries

We have repeated the experiment for the dataset of 1 mil. MPEG-7 descriptors with the same settings. That is, the buffer 8,000, up to 40,000 descriptors

in the cache, 15 min delay limit, and 100,000 queries processed with 1,000 candidate objects used for each query. The results are presented in Fig. 5b. We were able to reduce the processing time 3.72 times compared to the *base* approach. The cache utility of the *full* greedy algorithm was 0.66; its median delay was 79 s, and there were 169 timed out queries.

In Fig. 5c, we can see a comparison of processing times when the 10 mil. dataset of MPEG-7 descriptors is used. We used a buffer of size 10,000; up to 90,000 descriptors were stored in the cache (0.9 % of the DB); 100,000 queries were processed. The timeout limit was set to 30 min. Up to 2,000 candidate objects were used for each approximate query. The *full* greedy algorithm clearly outperforms the *base* approach as the processing was 2 times faster. The median delay of the *full* greedy algorithm was 548 s (9 min); the cache utility was 0.38; there were 2,838 timeouts.

Through the previous experiments, we verified the validity of our approach. That is, by finding long sequences of similar pairs of queries, we achieve high cache utility, and thus low processing time.

8 Conclusion

We have presented a novel approach to enhance the throughput of similarity search queries while obeying a given delay limit and processing every data item in the stream of queries. The technique is based on dynamic reordering of the incoming queries combined with appropriate caching strategy for partitions of the indexing structure. We have proposed three variants of the reordering by applying greedy heuristic approach to identify long sequences of similar queries. Two of the variants deal with starvation of the buffered queries by employing timeouts with different strategies of processing continuation. Our experimental evaluation proved our expectations and the most sophisticated technique was able to achieve up to 3.7 times higher throughput compared to the base case when no reordering is used.

Acknowledgements. This work was supported by the Czech national research project GA16-18889S.

References

1. Batko, M., Novak, D., Zezula, P.: MESSIF: metric similarity search implementation framework. In: Thanos, C., Borri, F., Candela, L. (eds.) Digital Libraries: Research and Development. LNCS, vol. 4877, pp. 1–10. Springer, Heidelberg (2007)
2. Budikova, P., Batko, M., Zezula, P.: Evaluation platform for content-based image retrieval systems. In: Gradmann, S., Borri, F., Meghini, C., Schuldt, H. (eds.) TPD 2011. LNCS, vol. 6966, pp. 130–142. Springer, Heidelberg (2011)
3. Falchi, F., Lucchese, C., Orlando, S., Perego, R., Rabitti, F.: Similarity caching in large-scale image retrieval. *Inf. Process. Manage.* **48**(5), 803–818 (2012)

4. Gaber, M.M., Zaslavsky, A., Krishnaswamy, S.: A survey of classification methods in data streams. In: Aggarwal, C.C. (ed.) *Data Streams: Models and Algorithms*. Advances in Database Systems, vol. 31, pp. 39–59. Springer, Heidelberg (2007)
5. Jia, Y., Shelhamer, E., Donahue, J., Karayev, S., Long, J., Girshick, R., Guadarrama, S., Darrell, T.: Caffe: convolutional architecture for fast feature embedding. In: *Proceedings of the ACM International Conference on Multimedia*, pp. 675–678. ACM (2014)
6. Law, Y.-N., Zaniolo, C.: An adaptive nearest neighbor classification algorithm for data streams. In: Jorge, A.M., Torgo, L., Brazdil, P.B., Camacho, R., Gama, J. (eds.) *PKDD 2005*. LNCS (LNAI), vol. 3721, pp. 108–120. Springer, Heidelberg (2005)
7. Mera, D., Batko, M., Zezula, P.: Towards fast multimedia feature extraction: hadoop or storm. In: *2014 IEEE International Symposium on Multimedia (ISM)*, pp. 106–109. IEEE (2014)
8. Nalepa, F., Batko, M., Zezula, P.: Performance analysis of distributed stream processing applications through colored petri nets. In: Kofron, J., Vojnar, T. (eds.) *MEMICS 2015*. LNCS, vol. 9548, pp. 93–106. Springer, Heidelberg (2016). doi:[10.1007/978-3-319-29817-7_9](https://doi.org/10.1007/978-3-319-29817-7_9)
9. Novak, D., Batko, M., Zezula, P.: Metric index: an efficient and scalable solution for precise and approximate similarity search. *Inf. Syst.* **36**(4), 721–733 (2011)
10. Pandey, S., Broder, A., Chierichetti, F., Josifovski, V., Kumar, R., Vassilvitskii, S.: Nearest-neighbor caching for content-match applications. In: *Proceedings of the 18th International Conference on World Wide Web*, pp. 441–450. ACM (2009)
11. Pietruczuk, L., Duda, P., Jaworski, M.: A new fuzzy classifier for data streams. In: Rutkowski, L., Korytkowski, M., Scherer, R., Tadeusiewicz, R., Zadeh, L.A., Zurada, J.M. (eds.) *ICAISC 2012, Part I*. LNCS, vol. 7267, pp. 318–324. Springer, Heidelberg (2012)
12. Shao, J., Huang, Z., Shen, H.T., Zhou, X., Lim, E.P., Li, Y.: Batch nearest neighbor search for video retrieval. *IEEE Trans. Multimed.* **10**(3), 409–420 (2008)
13. Tao, Y., Papadias, D., Shen, Q.: Continuous nearest neighbor search. In: *Proceedings of the 28th International Conference on Very Large Data Bases, VLDB Endowment*, pp. 287–298 (2002)
14. Zezula, P., Amato, G., Dohnal, V., Batko, M.: *Similarity Search: The Metric Space Approach*. Advances in Database Systems, vol. 32. Springer, Heidelberg (2006)
15. Zhang, P., Zhou, C., Wang, P., Gao, B.J., Zhu, X., Guo, L.: E-tree: an efficient indexing structure for ensemble models on data streams. *IEEE Trans. Knowl. Data Eng.* **27**(2), 461–474 (2015)